Clearspace Enhancement Narrative

Artifact Background and Selection

I created Clearspace in August 2026 as a Windows file manager using C# and Windows Presentation Foundation. The original application supported folder browsing, search, tags, and file operations. I selected it for my CS 499 ePortfolio because it gave me an opportunity to improve a working application across software design and engineering, algorithms and data structures, and databases. Each enhancement addresses a different part of the same user experience.

The portfolio preserves the original application and three stages of enhanced source. Artifact One contains the software engineering changes, Artifact Two builds on that work with algorithm improvements, and Artifact Three adds the database and file-protection work. These stages let a reader compare the application before and after each enhancement. My purpose was to make Clearspace easier to maintain and more dependable while learning how design choices affect its behavior.

Software Design and Engineering

The original MainViewModel coordinated too many responsibilities. I moved search handling into SearchCoordinator, folder loading into NavigationCoordinator, and sidebar state into SidebarViewModel. This made individual behaviors easier to understand and test without starting the full interface. I accepted the cost of having more classes because each class now has a clearer responsibility.

I also replaced Boolean file-operation results with FileOperationResult. Copy, move, rename, and delete can now report success, cancellation, or failure, together with Windows error information. Updating the callers mattered as much as changing the result type: canceling a copy

should not produce the same feedback as a permission error. These changes demonstrate refactoring, interface design, error handling, and designing components for testing.

The hardest lesson involved background searches. Canceling an old search did not by itself prevent late results from replacing a newer search. Separating the coordinator from its sources let me reproduce that timing problem with controlled tests. Testing also exposed a filter-only search that skipped subfolders and a cleared search that restored items without restoring the status message. Correcting those behaviors showed me why compilation and a visual walkthrough are not enough. The saved software-design verification passed 87 Release tests.

Algorithms and Data Structures

Before this enhancement, indexing stopped at 32 folder levels and fallback search stopped at 24. The same location could therefore produce different results depending on the search path. I removed the arbitrary depth limits while retaining cancellation and safe handling of folder links. Indexing uses an explicit stack, and fallback search uses queues. Testing with a 48-level folder structure helped verify that the improvement addressed the original coverage problem.

Folder sizes are calculated in a bottom-up pass over the indexed parent relationships. Each completed subtree total is added to its parent once, requiring O(n) work rather than O(n times d) work to walk every entry's ancestors, where d is folder depth. Arrays keep totals and hierarchy information compact. The disk-usage viewer caches layouts and skips off-screen branches, while per-drive index health allows healthy drives to remain useful when another drive needs recovery.

This work taught me to evaluate speed and correctness together. Deeper traversal increases work, and caches require memory limits and rules for stale data. Dense folders exposed

freezing and flickering; at one point, labels moved without their blocks. I learned that the camera, displayed geometry, and click targets must describe the same frame. I also improved indexing feedback because a percentage could decrease as more folders were discovered. Explaining waiting states and retry options made the underlying algorithm easier for users to understand. The saved algorithms verification passed 158 tests, but that result does not establish smooth performance on every drive.

Databases and File Protection

The original tag store rewrote a JSON document after each change and maintained relationships in application code. I replaced it with SQLite tables for tags, paths, and assignments. Keys and constraints prevent duplicate or orphaned relationships, an index supports tag lookups, and transactions keep related changes together. Parameterized queries keep user values separate from SQL instructions. The initial migration preserves the original JSON. I learned that a successful migration must preserve existing behavior as well as move data.

The database also stores lock metadata and password-verification information. Artifact Three extends this work with file and folder locking, temporary unlocking, automatic re-locking, password changes, and File Explorer integration. File protection uses AES-256-GCM and password-derived protection through PBKDF2. The database does not store plaintext passwords or unwrapped file keys. Each protected file carries its own recovery header, reducing dependence on a particular local database or filename.

The most important lesson was that protecting data includes planning for interrupted operations. A SQLite transaction cannot also commit a Windows file replacement. The implementation records recovery information, prepares and checks a replacement, changes the file, and then updates database state. Unlocking files in place lets other programs use them, but

also makes them readable during that visit. The documented local NTFS and file-size limits narrow the supported cases. Encryption also does not erase older copies in backups or caches.

Feedback and Revisions

One reported problem was that nothing happened when the feature was tried. Investigation showed that an older installed copy was running. That experience changed how I investigate problems: I check the running build before debugging the current source. I also abandoned an unreliable attempt to make File Explorer request a password before opening a locked folder and kept the simpler supported behavior.

My own testing and review led to further revisions. Reviewing password recovery exposed a design that checked the saved password before trying the protected file. That could refuse older files after a reset with a new password. The later design checks the file itself and offers to move it to the current password. These are examples of incorporating a reported issue and findings from self-review. They are distinct from instructor feedback and do not demonstrate that I developed Clearspace with a team.

Alignment With Course Outcomes

Outcome 1 concerns collaboration that supports decision making. Clearspace was an independent project. Focused components, implementation notes, and a code review make it easier for others to review decisions, but those materials alone do not prove collaborative development. My professional self-assessment provides the separate background of my IT support work and Unity collaboration. This artifact offers supporting preparation for collaboration rather than complete evidence for that outcome.

Outcome 2 is supported by the code review, narratives, and interface explanations. These communicate technical decisions and limitations to developers, instructors, and users. Outcome 3

is supported by traversal choices, bottom-up aggregation, cache rules, and the evaluation of memory and responsiveness tradeoffs. Outcome 4 is supported by applying C#, WPF, SQLite, and automated tests to working features. Outcome 5 is supported by parameterized queries, validation, authenticated encryption, recovery planning, and tests for wrong passwords and altered files.

Verification and Remaining Work

The earlier database snapshot passed 251 tests. The final report included with the supplied Artifact Three archive records 283 tests, with 280 passes and three failures. Those failures concern an older-password message assertion, cleanup of a locked-folder fixture, and index retry timing. This is evidence of remaining verification work, so I do not describe the final suite as fully passing. The earlier results and the later report refer to different snapshots.

Together, the enhancements show my growth from adding features to evaluating how the application behaves under load, cancellation, and failure. They also show why limitations belong in a professional portfolio. Resolving the remaining test failures and continuing checks on real drives would strengthen the final artifact. The original source, staged enhancements, and narratives provide a record of the design choices and learning behind that work.